Comparación experimental del rendimiento de Bubble Sort y Merge Sort bajo distintos escenarios de entrada

Experimental Comparison of Bubble Sort and Merge Sort Performance under Different Input Scenarios

Omar Alejandro Flores Anahua
Escuela Profesional de Ingeniería en Informática y Sistemas,
Universidad Nacional Jorge Basadre Grohmann, Tacna, Perú
ofloresa@unjbg.edu.pe

Resumen: ***C****omparar experimentalmente el rendimiento de Bubble Sort y Merge Sort, implementados en C++, bajo distintos tamaños y patrones de entrada. Se programaron ambos algoritmos sin funciones de ordenación de la biblioteca estándar y se evaluaron cinco patrones de entrada (aleatorio, ascendente, descendente, casi ordenado y con duplicados) con* $n = 1\,000$*,* $10\,000$ *y* $100\,000$*, con 30 repeticiones por condición y semillas fijas; Bubble Sort no se midió con* $n = 100\,000$ *por su costo de ejecución. Se registraron el tiempo, las comparaciones, los intercambios o movimientos y la profundidad de recursión. Con* $n = 10\,000$ *y datos aleatorios, la mediana del tiempo fue de 285,4 ms en Bubble Sort y de 1,75 ms en Merge Sort, unas 163 veces menor. Bubble Sort realizó siempre* $n(n-1)/2$ *comparaciones y su tiempo varió con los intercambios; Merge Sort tardó entre 10 y 20 ms con* $n = 100\,000$*, según el patrón. El crecimiento medido fue compatible con* $O(n^2)$ *y* $O(n \log n)$*, y el patrón de entrada modificó los tiempos en un factor cercano a 2 en ambos algoritmos. La ventaja de Merge Sort se obtuvo a cambio de memoria auxiliar de orden* $O(n)$*.*

Palabras clave: algoritmos de ordenamiento, Bubble Sort, Merge Sort, rendimiento, análisis experimental.

Abstract: ***T****o experimentally compare the performance of Bubble Sort and Merge Sort, implemented in C++, under different input sizes and data patterns. Both algorithms were written by hand without standard-library sorting functions and tested on five input patterns (random, ascending, descending, nearly sorted, and with duplicates) with* $n = 1{,}000$*,* $10{,}000$*, and* $100{,}000$*, using 30 repetitions per condition and fixed seeds; Bubble Sort was not measured at* $n = 100{,}000$ *because of its running cost. Execution time, comparisons, swaps or element moves, and recursion depth were recorded. For* $n = 10{,}000$ *and random data, the median time was 285.4 ms for Bubble Sort and 1.75 ms for Merge Sort, about 163 times lower. Bubble Sort always performed* $n(n-1)/2$ *comparisons and its time varied with the number of swaps; Merge Sort took between 10 and 20 ms at* $n = 100{,}000$*, depending on the pattern. The measured growth was consistent with* $O(n^2)$ *and* $O(n \log n)$*, and the input pattern changed the times by a factor close to 2 in both algorithms. Merge Sort's advantage came at the cost of* $O(n)$ *auxiliary memory.*

Keywords: sorting algorithms, Bubble Sort, Merge Sort, performance, experimental analysis.

1. Introducción

Los algoritmos de ordenamiento son procedimientos fundamentales en el procesamiento de datos, debido a que permiten organizar elementos de acuerdo con un criterio determinado. Su utilización facilita la búsqueda, el análisis y la presentación de información en

diferentes aplicaciones informáticas. Sin embargo, los algoritmos que resuelven el mismo problema pueden presentar diferencias importantes en su tiempo de ejecución, uso de memoria y comportamiento frente a distintos tipos de entrada.

Entre los algoritmos de ordenamiento se encuentran Bubble Sort y Merge Sort. Bubble Sort compara elementos adyacentes y los intercambia cuando se encuentran en un orden incorrecto. Su implementación es sencilla, aunque su complejidad temporal promedio y en el peor caso es cuadrática. Por otro lado, Merge Sort divide los datos en partes más pequeñas, ordena cada una y combina los resultados. Este procedimiento presenta una complejidad temporal de $O(n \log n)$ en sus casos mejor, promedio y peor, aunque necesita memoria auxiliar adicional para la combinación.

El rendimiento de estos algoritmos puede variar según la cantidad de elementos y el patrón de entrada utilizado. Las versiones de Bubble Sort con parada anticipada se benefician de las listas que ya están ordenadas, mientras que la versión canónica realiza el mismo número de comparaciones sin importar el patrón. Una lista desordenada de gran tamaño, por su parte, evidencia las diferencias entre las estrategias de ambos algoritmos. Por esta razón, resulta conveniente estudiar su comportamiento bajo diferentes condiciones experimentales.

La comparación empírica de algoritmos de ordenamiento ya ha sido abordada mediante análisis estadístico (Salgado et al., 2022) y mediante la medición de la eficiencia de implementaciones recursivas e iterativas (Murillo y Caamaño, 2013), mientras que la complejidad de los algoritmos clásicos de ordenamiento se encuentra descrita en textos de referencia (Cormen et al., 2022; Knuth, 1998).

El objetivo de este estudio es comparar experimentalmente Bubble Sort y Merge Sort, implementados en C++, considerando distintos tamaños y patrones de entrada. Para ello, se registraron el tiempo de ejecución, el número de comparaciones, los intercambios o movimientos de elementos y la profundidad máxima de recursión cuando corresponde.

Como expectativa descriptiva, se planteó que Bubble Sort presentaría un crecimiento del tiempo de ejecución más pronunciado que Merge Sort al aumentar el tamaño de la entrada, y que Merge Sort mostraría un crecimiento compatible con su complejidad $O(n \log n)$. Estas expectativas se contrastan con los datos obtenidos en la Discusión.

2. Metodología experimental

2.1. Selección de los algoritmos

Para realizar la comparación experimental se seleccionaron Bubble Sort y Merge Sort, debido a que utilizan estrategias diferentes para resolver el problema del ordenamiento. Bubble Sort se basa en comparaciones e intercambios entre elementos adyacentes, mientras que Merge Sort emplea la estrategia de divide y vencerás. Esta selección permite estudiar las diferencias entre un algoritmo de implementación sencilla y otro que divide recursivamente el problema en partes más pequeñas. Ambos se programaron a mano en C++, sin emplear funciones de la biblioteca estándar para ordenar, copiar o intercambiar elementos.

2.1.1. Bubble Sort

Se utilizó la versión canónica de Bubble Sort, sin condición de terminación anticipada. En la pasada i se recorre el arreglo de izquierda

a derecha, se compara cada elemento $A[j]$ con el siguiente y se intercambian cuando $A[j] > A[j+1]$, de modo que el mayor elemento de la parte no ordenada queda al final.

Como no existe parada anticipada, el algoritmo realiza siempre $n(n-1)/2$ comparaciones, por lo que su complejidad temporal es $O(n^2)$ en los casos mejor, promedio y peor. Una variante con parada anticipada tendría $O(n)$ en el mejor caso, pero no se empleó en este estudio. Su memoria auxiliar es $O(1)$, debido a que ordena los elementos en el arreglo original.

La facilidad de implementación de Bubble Sort permite utilizarlo como referencia para estudiar el efecto del patrón de entrada sobre el rendimiento. No obstante, su número de comparaciones e intercambios aumenta considerablemente con el tamaño de la entrada.

2.1.2. Merge Sort

Merge Sort se implementó mediante una variante recursiva descendente basada en la estrategia de divide y vencerás. El procedimiento divide el arreglo en dos mitades, ordena recursivamente cada una y combina ambas en una secuencia ordenada, usando un arreglo auxiliar de n elementos.

Su complejidad temporal es $O(n \log n)$ en los casos mejor, promedio y peor, y su memoria auxiliar es $O(n)$. La profundidad máxima de recursión se registró como el número de niveles de llamadas, contando el nivel inicial.

Aunque Merge Sort requiere una implementación más elaborada que Bubble Sort, su estructura permite abordar entradas grandes con un crecimiento temporal teórico menor. La utilización de memoria adicional constituye una diferencia importante que debe tenerse en cuenta al comparar ambos algoritmos.

Tabla 1. Características teóricas de los algoritmos.

	Bubble Sort	**Merge Sort**
Estrategia	Intercambios adyacentes	Divide y vencerás
Mejor caso	$O(n^2)$	$O(n \log n)$
Caso promedio	$O(n^2)$	$O(n \log n)$
Peor caso	$O(n^2)$	$O(n \log n)$
Memoria auxiliar	$O(1)$	$O(n)$
Estabilidad	Estable	Estable
Recursión	No	Sí

2.1.3. Complejidad y memoria

La Tabla 1 presenta las características teóricas de las variantes utilizadas. Estas propiedades se emplearon como referencia para interpretar los resultados experimentales, sin sustituir las mediciones reales. La memoria corresponde al espacio auxiliar adicional requerido por cada implementación.

2.1.4. Estabilidad

La estabilidad es la propiedad que permite conservar el orden relativo original de los elementos que poseen la misma clave. Esta característica puede ser importante cuando se ordenan registros que contienen varios campos.

En Bubble Sort solo se intercambian elementos cuando $A[j] > A[j+1]$, es decir, nunca se intercambian claves iguales. En Merge Sort, durante la combinación, se toma primero el elemento de la mitad izquierda cuando ambas claves son iguales ($A[i] \leq A[j]$). Por ello, ambas implementaciones son estables. Esta propiedad se dedujo del código y no se verificó experimentalmente, porque los elementos ordenados fueron enteros simples, que no permiten distinguir claves iguales.

Tabla 2. Patrones de entrada utilizados.

Patrón	Descripción
Aleatorio	Enteros uniformes entre 1 y 1 000 000.
Ascendente	$1, 2, \ldots, n$.
Descendente	$n, n-1, \ldots, 1$.
Casi ordenado	Arreglo ascendente con $n/40$ intercambios aleatorios (cerca de 5 % de los elementos movidos).
Con duplicados	Quince valores únicos, entre 1 y 15, repetidos.

2.2. Patrones de entrada

Se utilizaron cinco patrones de entrada, descritos en la Tabla 2, para estudiar no solo el caso de una lista desordenada al azar, sino también el comportamiento de los algoritmos cuando los datos ya presentan cierto orden o valores repetidos.

2.3. Tamaños, repeticiones y semillas

Se consideraron tres tamaños de entrada, $n \in \{1\,000;\ 10\,000;\ 100\,000\}$. Cada combinación de algoritmo, patrón y tamaño se repitió 30 veces, excepto Bubble Sort con $n = 100\,000$, que se omitió. En ese tamaño, Bubble Sort realiza cerca de 5×10^9 comparaciones por ejecución, lo que vuelve poco práctico repetirlo 30 veces por patrón. Por ello su evaluación se limitó a $n = 1\,000$ y $n = 10\,000$, mientras que Merge Sort se evaluó con los tres tamaños. En total se obtuvieron 750 ejecuciones: 300 de Bubble Sort y 450 de Merge Sort.

Para facilitar la repetición del experimento se utilizó un generador congruencial lineal propio, en lugar de `rand()`, de modo que los datos son idénticos en cualquier equipo y compilador. La semilla base fue 12345 y la repetición r usó la semilla $12345 + r$, es decir, de 12346 a 12375. En cada repetición, ambos algoritmos recibieron exactamente los mismos datos de entrada.

2.4. Métricas y registro de datos

Se registraron cinco medidas por ejecución: el tiempo de ordenamiento en milisegundos, medido con `std::chrono::steady_clock`; el número de comparaciones entre elementos; los intercambios, que en Bubble Sort son intercambios de pares y en Merge Sort son los movimientos de elementos al arreglo auxiliar, porque este algoritmo no intercambia como tal; y la profundidad máxima de recursión, que solo se aplica a Merge Sort y vale 0 en Bubble Sort.

El tiempo se midió únicamente durante el ordenamiento: la generación de los datos, la verificación del resultado y la escritura del archivo quedaron fuera del intervalo cronometrado. Cada fila del archivo CSV corresponde a una ejecución e incluye, además, el algoritmo, el tamaño, el patrón, el número de repetición, la semilla y el identificador de la versión del código. Después de cada ejecución se comprobó que el arreglo quedara ordenado y no se detectó ningún error en las 750 ejecuciones. Los contadores forman parte del trabajo realizado dentro de los algoritmos medidos, por lo que añaden un pequeño costo al tiempo, igual para ambos.

2.5. Entorno de ejecución

Las pruebas se realizaron en un equipo con procesador Intel Core i5-7400 a 3,00 GHz (4 núcleos y 4 hilos), 8 GB de memoria RAM DDR4 a 2133 MHz y sistema operativo Windows 11. El código se compiló con TDM-GCC 4.9.2 de 64 bits desde Dev-C++, con las opcion `-static-libgcc` y sin indicadores de

optimización (`-O`). Durante la corrida final se cerraron los demás programas. Los tiempos obtenidos corresponden a este equipo y pueden variar en otro.

2.6. Análisis de los datos

Para cada algoritmo, patrón y tamaño se calcularon la media, la mediana, la desviación estándar, el mínimo y el máximo del tiempo, y el promedio de los contadores, con un script de Python (`pandas` y `matplotlib`). El código, los datos y el script se encuentran en el repositorio del proyecto, y los resultados corresponden al código del commit `2c4e2dc`.

3. Resultados

3.1. Tiempos de ejecución

La Tabla 3 y la Tabla 4 resumen los tiempos de Bubble Sort y de Merge Sort. Con $n = 10\,000$, la mediana de Bubble Sort estuvo entre 125,5 ms (ascendente) y 286,9 ms (con duplicados), mientras que la de Merge Sort estuvo entre 0,85 ms (ascendente) y 1,75 ms (aleatorio). Con datos aleatorios, la mediana de Bubble Sort fue unas 163 veces mayor que la de Merge Sort. Con $n = 1\,000$ esa razón fue de unas 17 veces.

Con $n = 100\,000$ solo se midió Merge Sort, cuyas medianas estuvieron entre 10,4 ms (ascendente) y 20,0 ms (aleatorio).

3.2. Crecimiento con el tamaño y variabilidad

La Figura 1 muestra el tiempo mediano en función de n. Al pasar de $n = 1\,000$ a $n = 10\,000$, la mediana de Bubble Sort se multiplicó entre 102 y 140 veces, según el patrón. En Merge Sort, el mismo cambio multiplicó la mediana entre 11,8 y 14,2 veces, y el paso de 10 000 a 100 000 la multiplicó entre 11,4 y 12,3 veces.

La Figura 2 presenta la distribución de los 30 tiempos con $n = 10\,000$. En Bubble Sort, el coeficiente de variación estuvo entre 2,3 % y 23,6 % con $n = 10\,000$. En Merge Sort fue mayor con $n = 1\,000$ (de 13,2 % a 98,6 %) y menor con $n = 100\,000$ (de 2,3 % a 14,1 %). También se observaron valores atípicos aislados, como un máximo de 524 ms en Bubble Sort descendente con $n = 10\,000$, frente a una mediana de 222 ms.

Tabla 3. Estadística descriptiva del tiempo de ejecución (ms) de Bubble Sort, con 30 repeticiones por condición.

Patrón	n	Media	Mediana	Desv. est.	Mín.	Máx.
Aleatorio	1 000	2,21	2,13	0,17	2,06	2,83
Aleatorio	10 000	295,97	285,38	24,92	278,61	404,05
Ascendente	1 000	1,26	1,23	0,09	1,18	1,62
Ascendente	10 000	126,56	125,46	2,87	123,92	133,92
Descendente	1 000	2,17	2,13	0,09	2,07	2,42
Descendente	10 000	235,82	222,02	55,57	217,95	524,37
Casi ordenado	1 000	1,51	1,38	0,42	1,32	3,54
Casi ordenado	10 000	156,48	147,69	19,35	140,25	207,74
Con duplicados	1 000	2,13	2,05	0,19	2,01	2,79
Con duplicados	10 000	300,35	286,85	28,04	276,77	393,21

Tabla 4. Estadística descriptiva del tiempo de ejecución (ms) de Merge Sort, con 30 repeticiones por condición.

Patrón	n	Media	Mediana	Desv. est.	Mín.	Máx.
Aleatorio	1 000	0,166	0,128	0,139	0,116	0,868
Aleatorio	10 000	1,869	1,754	0,322	1,628	3,195
Aleatorio	100 000	20,211	19,975	0,603	19,708	22,615
Ascendente	1 000	0,069	0,061	0,017	0,060	0,119
Ascendente	10 000	0,880	0,848	0,094	0,792	1,223
Ascendente	100 000	10,642	10,400	0,636	10,167	13,072
Descendente	1 000	0,106	0,074	0,105	0,061	0,589
Descendente	10 000	0,939	0,880	0,144	0,805	1,365
Descendente	100 000	10,935	10,479	1,544	10,132	18,408
Casi ordenado	1 000	0,069	0,066	0,009	0,065	0,112
Casi ordenado	10 000	0,968	0,930	0,103	0,880	1,335
Casi ordenado	100 000	11,363	11,306	0,264	10,996	12,031
Con duplicados	1 000	0,110	0,101	0,029	0,099	0,253
Con duplicados	10 000	1,275	1,240	0,123	1,189	1,811
Con duplicados	100 000	15,251	14,743	1,520	14,485	22,694

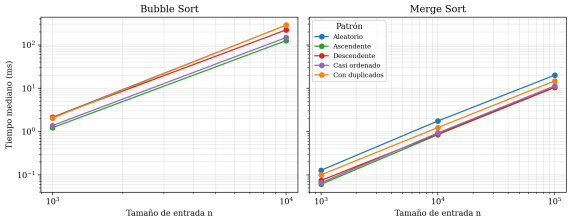


Figura 1. Tiempo mediano de ejecución en función de n, en escala logarítmica en ambos ejes.

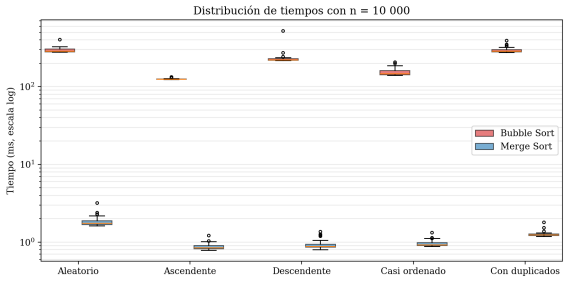


Figura 2. Distribución de los tiempos de ejecución con $n = 10\,000$ (escala logarítmica).

3.3. Comparaciones, intercambios y profundidad

La Tabla 5 muestra que Bubble Sort realizó siempre $n(n-1)/2$ comparaciones: 499 500 con $n = 1\,000$ y 49 995 000 con $n = 10\,000$, sin importar el patrón. Lo que cambió con el patrón fue el número de intercambios: 0 en el patrón ascendente, $n(n-1)/2$ en el descendente y cerca de la mitad de esa cantidad en los patrones aleatorio y con duplicados.

En Merge Sort (Tabla 6) los movimientos al arreglo auxiliar fueron iguales en todos los patrones para cada tamaño (9 976, 133 616 y 1 668 928), mientras que las comparaciones dependieron del patrón: con $n = 10\,000$ fueron 120 457 en el aleatorio y 64 608 en el descendente. La profundidad de recursión fue 11, 15 y 18 para $n = 1\,000$, 10 000 y 100 000. La Figura 3 resume estos contadores con $n = 10\,000$.

Tabla 5. Contadores de Bubble Sort (promedio de 30 repeticiones). La profundidad de recursión es 0.

Patrón	n	Comparaciones	Intercambios
Aleatorio	1 000	499 500	249 025
Aleatorio	10 000	49 995 000	25 015 115
Ascendente	1 000	499 500	0
Ascendente	10 000	49 995 000	0
Descendente	1 000	499 500	499 500
Descendente	10 000	49 995 000	49 995 000
Casi ordenado	1 000	499 500	16 656
Casi ordenado	10 000	49 995 000	1 623 316
Con duplicados	1 000	499 500	233 856
Con duplicados	10 000	49 995 000	23 363 148

Tabla 6. Contadores de Merge Sort (promedio de 30 repeticiones). Los movimientos son copias al arreglo auxiliar.

Patrón	n	Comparaciones	Movimientos	Profundidad
Aleatorio	1 000	8 712	9 976	11
Aleatorio	10 000	120 457	133 616	15
Aleatorio	100 000	1 536 397	1 668 928	18
Ascendente	1 000	5 044	9 976	11
Ascendente	10 000	69 008	133 616	15
Ascendente	100 000	853 904	1 668 928	18
Descendente	1 000	4 932	9 976	11
Descendente	10 000	64 608	133 616	15
Descendente	100 000	815 024	1 668 928	18
Casi ordenado	1 000	7 163	9 976	11
Casi ordenado	10 000	107 131	133 616	15
Casi ordenado	100 000	1 403 802	1 668 928	18
Con duplicados	1 000	8 588	9 976	11
Con duplicados	10 000	118 268	133 616	15
Con duplicados	100 000	1 502 989	1 668 928	18

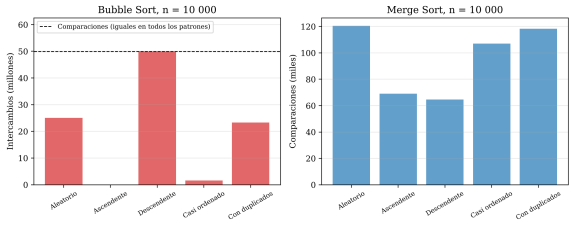


Figura 3. Intercambios de Bubble Sort y comparaciones de Merge Sort con $n = 10\,000$ (promedio de 30 repeticiones). La línea discontinua marca las comparaciones de Bubble Sort.

4. Discusión

4.1. Relación con la complejidad teórica

Los crecimientos medidos son compatibles con las complejidades teóricas. Para Bubble Sort, el número de comparaciones coincide exactamente con $n(n-1)/2$ y, al multiplicar n por 10, la teoría cuadrática predice un tiempo unas 100 veces mayor; las medianas crecieron entre 102 y 140 veces, un poco más en los patrones aleatorio y con duplicados. Para Merge Sort, la teoría $O(n \log n)$ predice un factor de 13,3 al pasar de 1 000 a 10 000 y de 12,5 al pasar de 10 000 a 100 000, y los valores medidos (11,8 a 14,2 y 11,4 a 12,3) estuvieron cerca. La expectativa descriptiva planteada en la introducción se cumplió: el crecimiento de Bubble Sort fue más pronunciado y el de Merge Sort fue compatible con $O(n \log n)$.

4.2. Efecto del patrón de entrada

En Bubble Sort, las comparaciones fueron fijas, por lo que las diferencias entre patrones provienen de los intercambios y del comportamiento del procesador. El patrón ascendente no realiza intercambios y fue el más rápido (125,5 ms con $n = 10\,000$). Sin embargo, el patrón descendente, con el doble de intercambios que el aleatorio, fue más rápido que este (222,0 ms frente a 285,4 ms). Esto sugiere que, además del número de intercambios, influye la regularidad de las comparaciones, por ejemplo la predicción de saltos del procesador; esta explicación no se verificó con mediciones adicionales.

En Merge Sort, los patrones ascendente, descendente y casi ordenado requirieron menos comparaciones (entre 64 608 y 107 131 con $n = 10\,000$) que el aleatorio (120 457) y tardaron menos (de 0,85 a 0,93 ms frente a 1,75 ms). El patrón con duplicados tuvo un número de comparaciones parecido al aleatorio (118 268) pero un tiempo menor (1,24 ms), lo que de nuevo indica que el tiempo no depende solo del número de comparaciones.

4.3. Duplicados y estabilidad

Como ambas implementaciones son estables, los valores repetidos no alteran su orden relativo. En términos de rendimiento, con duplicados Bubble Sort hizo algo menos de intercambios que con datos aleatorios (23,4 millones frente a 25,0 millones con $n = 10\,000$), porque no intercambia claves iguales, y Merge Sort hizo menos comparaciones (118 268 frente a 120 457).

4.4. Memoria

Merge Sort necesitó un arreglo auxiliar del mismo tamaño que la entrada, que con $n = 100\,000$ enteros de 4 bytes equivale a unos 0,4 MB, mientras que Bubble Sort ordenó en el sitio. La ventaja de tiempo de Merge Sort se obtuvo, por tanto, a cambio de memoria adicional. La memoria no se midió experimentalmente; se consideró el valor teórico.

4.5. Limitaciones y amenazas a la validez

Los resultados provienen de un solo equipo, un solo compilador con su configuración por defecto y datos sintéticos de enteros, por lo que no deben interpretarse como tiempos universales. Bubble Sort no se midió con $n = 100\,000$, de modo que la comparación directa entre los dos algoritmos se limita a $n = 1\,000$ y $n = 10\,000$. Con $n = 1\,000$, Merge Sort tarda unas décimas de milisegundo y su

variabilidad relativa fue alta (hasta 98,6 %), por lo que se emplearon las medianas. Los contadores añaden un costo a los tiempos y la estabilidad se dedujo del código sin verificarse experimentalmente. Por último, los valores atípicos pueden deberse a la actividad del sistema operativo.

5. Conclusiones

1. Con $n = 10\,000$ y datos aleatorios, la mediana fue de 285,4 ms en Bubble Sort y de 1,75 ms en Merge Sort, unas 163 veces menor, y con $n = 100\,000$ Merge Sort tardó entre 10 y 20 ms.
2. Bubble Sort realizó siempre $n(n-1)/2$ comparaciones, mientras que Merge Sort realizó entre 64 608 y 120 457 con $n = 10\,000$, según el patrón.
3. El crecimiento medido fue compatible con $O(n^2)$ en Bubble Sort y con $O(n \log n)$ en Merge Sort.
4. El patrón de entrada modificó los tiempos en un factor cercano a 2 en ambos algoritmos, y esa variación no se explica solo por el número de comparaciones o intercambios.
5. La ventaja de tiempo de Merge Sort requiere un arreglo auxiliar de orden $O(n)$, mientras que Bubble Sort usa memoria auxiliar constante.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Knuth, D. E. (1998). *The art of computer programming: Vol. 3. Sorting and searching* (2.ª ed.). Addison-Wesley.

Murillo, J., & Caamaño, S. (2013). Comparación entre algoritmos recursivos e iterativos y su medición en términos de eficiencia. *Uniciencia, 27*(1), 341–350. https://www.redalyc.org/pdf/4759/475947762020.pdf

Salgado, M., Pérez, M., & Albarrán, S. (2022). Algoritmos de ordenamiento, un análisis estadístico comparativo. *Ciencia e Ingeniería, 43*(3), 273–280. https://doi.org/10.53766/cei/2022.43.03.05